

INFORME FINAL – DESAFÍO 1

Implementación de Sweet Crush mediante manipulación de bits en C++

INFORMÁTICA II – 2026-2

YEIMMY YISSEL CUESTA JIMÉNEZ

JULIANA MENDEZ MOZQUERA

1. Contextualización

El Desafío 1 consiste en diseñar e implementar una solución del juego Sweet Crush utilizando manipulación de bits, memoria dinámica y programación estructurada en C++. El tablero debe almacenarse como una secuencia compacta de bits, utilizando exactamente tres bits por posición. Además, el programa debe permitir eliminar fichas, detectar combinaciones, reorganizar el tablero, modificar filas y columnas, aplicar gravedad, generar nuevas fichas y procesar cascadas.

En comparación con el informe preliminar, la solución evolucionó desde la definición de los módulos y los problemas principales hacia una integración mediante un programa principal que coordina las operaciones del tablero y las reglas del juego.

2. Cambios y mejoras respecto al informe preliminar

Integración del programa principal: Se incorporó un main.cpp que conecta los módulos de tablero, marcador, combinaciones, eliminación, gravedad, generación, estadísticas y conversión de posiciones.

Manipulación de bits: Se implementaron funciones para calcular la cantidad de bytes, localizar los tres bits correspondientes a una ficha y leer o modificar fichas incluso cuando sus bits atraviesan el límite de un byte.

Marcador de combinaciones: Se utiliza un arreglo dinámico de bits para marcar las posiciones que pertenecen a combinaciones antes de realizar la eliminación.

Combinaciones simultáneas: La búsqueda horizontal y vertical utiliza el mismo marcador, permitiendo conservar la información de posiciones que participan en más de una combinación.

Eliminación y gravedad: La eliminación convierte las posiciones seleccionadas en vacías y la gravedad compacta cada columna desplazando las fichas hacia las posiciones inferiores.

Generación de fichas: Después de aplicar gravedad se rellenan los espacios vacíos con nuevas fichas aleatorias.

Cascadas: El procesamiento se repite mientras sigan apareciendo combinaciones después de una eliminación, una modificación estructural o la generación de fichas.

Memoria dinámica: El tamaño del tablero se calcula según el número real de posiciones y se contempla la reorganización de memoria al modificar filas o columnas.

Estadísticas: Se incorporó el registro de movimientos, fichas eliminadas, combinaciones, cascadas y puntuación.

3. Problemas de desarrollo y resolución

Fichas repartidas entre dos bytes

Como cada ficha ocupa tres bits y el byte contiene ocho, no siempre existe coincidencia entre el límite de una ficha y el límite de un byte. La solución consiste en calcular el desplazamiento físico y, cuando es necesario, dividir la lectura o escritura entre dos bytes, conservando los bits vecinos.

Reorganización al cambiar filas o columnas

Al insertar o eliminar una fila o columna cambia la cantidad de posiciones y la ubicación lógica de muchas fichas. ReorganizarTablero construye la nueva distribución recorriendo las nuevas posiciones y determinando qué posición anterior corresponde a cada una.

Combinaciones horizontales y verticales simultáneas

La detección horizontal y vertical escribe sobre un marcador común. De esta forma, una posición que pertenece a dos combinaciones continúa marcada una sola vez y no se elimina dos veces.

Cascadas

Después de eliminar las posiciones marcadas, se aplica gravedad y se generan nuevas fichas. El programa vuelve a buscar combinaciones y repite el procedimiento hasta que la búsqueda retorna cero combinaciones.

Control de memoria

La memoria se reserva dinámicamente de acuerdo con las posiciones actuales. En operaciones de reducción se contempla el criterio del 65 % definido para decidir cuándo reducir físicamente el bloque.

Validación de entradas

La función leerEntero controla errores de entrada y verifica que los valores estén dentro de los rangos permitidos antes de continuar.

4. Diseño final del sistema

La solución queda dividida en módulos especializados:

bits.h / bits.cpp: representación compacta de las fichas y operaciones de lectura/escritura a nivel de bits.

tablero.h / tablero.cpp: creación, limpieza, liberación y reorganización dinámica del tablero.

posicion.h / posicion.cpp: conversión entre coordenadas de fila/columna y posición lineal.

marcador.h / marcador.cpp: marcado y consulta de posiciones involucradas en combinaciones.

combinaciones.h / combinaciones.cpp: búsqueda horizontal, vertical y total de combinaciones.

eliminacion.h / eliminacion.cpp: eliminación de fichas y posiciones marcadas.

gravedad.h / gravedad.cpp: desplazamiento y compactación de las columnas.

generacion.h / generacion.cpp: generación aleatoria y llenado de posiciones vacías.

estadisticas.h / estadisticas.cpp: registro de movimientos, eliminaciones, combinaciones, cascadas y puntuación.

main.cpp: coordinación del flujo del juego y comunicación con el usuario.

5. Representación de las fichas

Código	Constante	Representación
000	F_CUADRADO	Cuadrado
001	F_HEXAGONO	Hexágono
010	F_RECTANGULO	Rectángulo
011	F_TRIANGULO	Triángulo
100	F_ROMBO	Rombo
101	F_PENTAGONO	Pentágono
110	F_VACIO	Posición vacía
111	F_ESPECIAL	Estado especial

La representación utiliza exactamente tres bits por posición. Los códigos 110 y 111 se aprovechan para representar estados adicionales: vacío y especial.

6. Gestión de memoria y representación física

El tablero se almacena mediante un puntero unsigned char*. El número de bytes se obtiene a partir de la cantidad de posiciones multiplicada por tres bits, redondeando hacia arriba al siguiente byte cuando es necesario. Los bits válidos se mantienen como una trama compacta y los bits de relleno se manejan separadamente.

La función ReorganizarTablero permite insertar y eliminar filas o columnas en posiciones válidas, incluidas posiciones intermedias. Durante la reorganización se conserva la información que debe permanecer en el tablero y las nuevas posiciones se inicializan como vacías.

7. Flujo funcional final

El usuario selecciona las dimensiones iniciales del tablero.

Se reserva la memoria necesaria y se inicializan marcador y estadísticas.

Se generan las fichas iniciales.

Se muestra el estado del tablero y las estadísticas.

El usuario selecciona una operación del menú.

Si elimina una ficha, se registra el movimiento y se procesa el resultado.

Si modifica una fila o columna, se reorganiza la estructura y se verifica nuevamente el tablero.

Se buscan combinaciones horizontales y verticales.

Las posiciones encontradas se marcan.

Se eliminan las fichas marcadas.

Se aplica gravedad.

Se generan nuevas fichas.

Se vuelve a buscar combinaciones y el ciclo continúa mientras existan nuevas combinaciones.

Al finalizar, se actualizan las estadísticas y se continúa con la siguiente jugada.

Figura 1. Representación en pseint.

8. Estadísticas y puntuación

El módulo de estadísticas utiliza un arreglo de enteros con posiciones definidas mediante constantes.

Se registran movimientos, fichas eliminadas, combinaciones, cascadas, cascadas de la jugada actual y puntuación. En la implementación revisada, la puntuación está definida como 10 puntos por ficha eliminada y 50 puntos por cascada.

9. Estado de integración del main.cpp

El main.cpp suministrado integra la creación del juego, el menú, las jugadas de eliminación y las modificaciones estructurales, además del procesamiento automático de cascadas. Sin embargo, antes de considerar la entrega completamente verificada, deben comprobarse las funciones utilizadas por el main que no aparecen en los encabezados revisados: reiniciarMarcador, mostrarTableroFichas y mostrarTableroBinario. También debe verificarse que sus declaraciones y definiciones estén incluidas en el proyecto Qt y que el proyecto compile sin errores.

Por lo tanto, el diseño lógico final está integrado, pero la validación final debe hacerse mediante compilación y pruebas del proyecto completo.

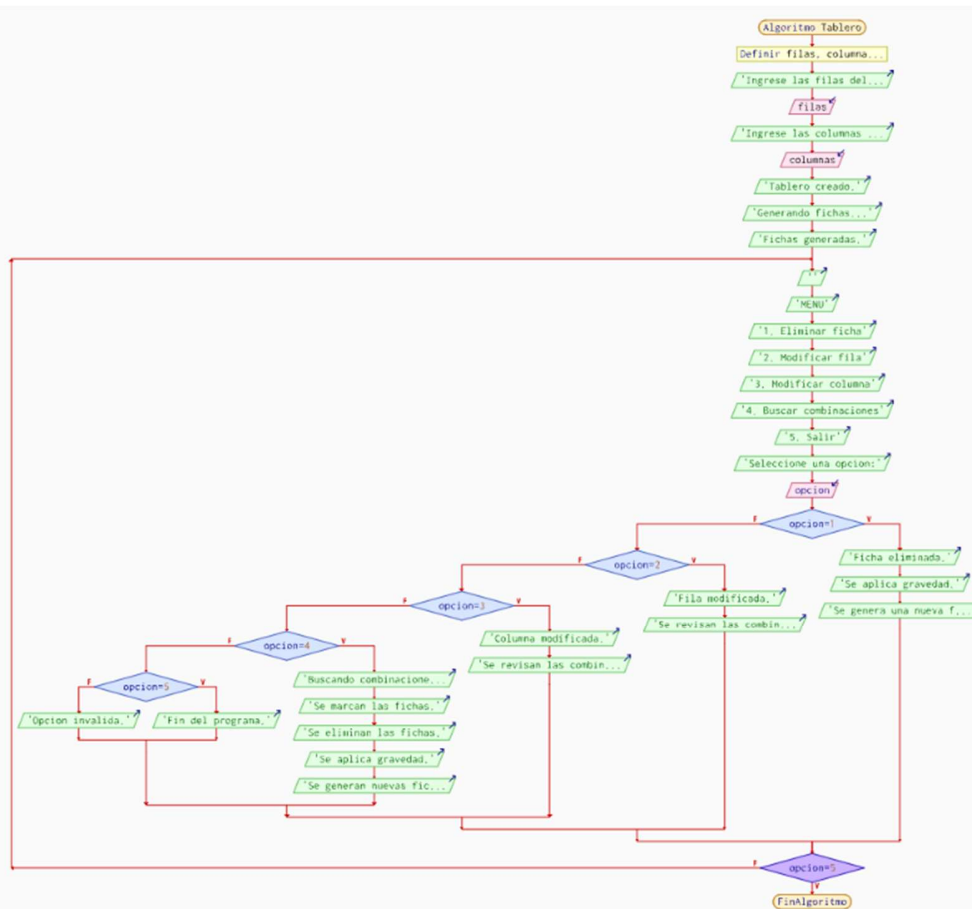


Figura II.

Compromisos del 19 de septiembre de 2026	diseño de principal: menú de jugadas, cascadas y visualización del tablero + cambios estéticos julsméndez-m comprometido ayer	a994a09	 
Confirmado el 16 de septiembre de 2026	Combinaciones y eliminacion con marcador de bits, gravedad y generacion de fichas julsméndez-m comprometido hace 4 días	9266e96	 
	Tablero: insertar y eliminar filas y columnas con regla del 65% julsméndez-m comprometido hace 4 días	5b22372	 
	Bits: relleno calculado con bytes reservados y limpieza del relleno julsméndez-m comprometido hace 4 días	8065223	 
Confirmado el 14 de septiembre de 2026	Módulos bits, tablero, posición, marcador y combinaciones julsméndez-m comprometido la semana pasada	cee5994	 
Compromisos del 11 de septiembre de 2026	informe desafio julsméndez-m comprometido la semana pasada	0d01286	 

Nota: Commits realizados durante el desarrollo del proyecto.

Evolución de la Solución y Proceso de Desarrollo (Basado en Commits)

El desarrollo de la solución se llevó a cabo de manera modular e incremental, utilizando un repositorio Git para documentar la evolución del código. A continuación, se detalla el proceso de implementación basado en los hitos (commits) realizados:

1. Fase de Análisis y Diseño (11 de septiembre)

- **Entrega de Documentación:** Se realizó el primer commit correspondiente al informe preliminar del desafío. En esta etapa se definió la contextualización del problema, el análisis de los requisitos (restricciones de memoria, uso de bits, dimensiones mínimas) y el diseño preliminar de la arquitectura del sistema.

2. Fase de Estructuración Modular (14 de septiembre)

- **Creación de Módulos Base:** Se implementó la arquitectura multi-archivo (.h y .cpp) requerida para evitar el uso de ANSI C y mantener el código organizado. Se crearon los módulos iniciales para:
 - La gestión de operaciones a nivel de bits.
 - La estructura del tablero.
 - El manejo de posiciones y coordenadas.
 - La lógica del marcador (puntaje).
 - El sistema de combinaciones y detección de colisiones iniciales.

3. Fase de Implementación del Núcleo Lógico (16 de septiembre)

- **Gestión de Memoria y Bits:** Se implementó el cálculo dinámico del tamaño de los arreglos basándose en el ancho del tablero (múltiplo de 8), utilizando bytes reservados. Se programó la lógica para la limpieza y el relleno de bits, optimizando el almacenamiento según lo exigido.
- **Operaciones sobre el Tablero:** Se desarrollaron los algoritmos para insertar y eliminar filas y columnas, aplicando las reglas de validación dimensional (incluyendo la regla del 65% para el cálculo del área de juego).
- **Mecánicas del Juego:** Se implementó el núcleo funcional del Tetris.
 - Detección de colisiones mediante el uso de máscaras de bits (evitando el uso de estructuras complejas).
 - Eliminación de filas completas cuando todos sus bits estaban en 1.
 - Aplicación de la gravedad (descenso de piezas).
 - Generación aleatoria uniforme de las fichas (tetriminos).

4. Fase de Interfaz y Refinamiento (19 de septiembre)

- **Interfaz de Usuario en Consola:** Se implementó el módulo principal (main), el cual incluye:
 - El menú interactivo de jugadas (Izquierda, Derecha, Bajar, Rotar, Salir).
 - La visualización del tablero en consola, traduciendo la representación binaria interna a caracteres visuales (# para ocupado, . para vacío).
 - La lógica de "cascadas" o flujo de turnos del juego.
- **Pulido Estético:** Se realizaron ajustes finales en la presentación visual del tablero y los mensajes al usuario para mejorar la experiencia de juego, asegurando el cumplimiento de los criterios de evaluación de la interfaz.

5. Fase de Integración Visual y Codificación Final (20 de septiembre)

- **Visualización y Codificación UTF-8:** Se realizó el commit final correspondiente a la integración completa de la estructura del tablero con la visualización de las fichas, respetando la figura definida en los requisitos. Se implementó la codificación UTF-8 para garantizar la correcta representación de los caracteres en consola. Además, se aplicó la lógica de código de bits sobre cada ficha, asegurando que la representación visual sea un reflejo exacto de su estado binario interno, cerrando así el ciclo de desarrollo del motor.

6. Conclusiones

El desarrollo de este desafío permitió llevar a la práctica los conceptos más avanzados de la programación en C++, demostrando que es posible construir un motor de juego funcional (Sweet Crush) bajo restricciones estrictas de eficiencia, memoria y estructuras de datos.

A lo largo de las distintas fases de implementación, se logró cumplir con la totalidad de los objetivos propuestos:

1. **Dominio de Operaciones a Nivel de Bits:** Se comprendió y aplicó la importancia de la representación binaria. El uso de máscaras (&), desplazamientos (<<, >>) y operadores OR (|) permitió extraer, modificar y escribir fichas de exactamente 3 bits dentro de una secuencia continua, incluso cuando estos bits atravesaban el límite de un byte. Esto demostró ser una técnica altamente eficiente para reducir el consumo de memoria.
2. **Gestión Responsable de la Memoria:** La implementación de memoria dinámica y punteros (unsigned char*) fue fundamental para adaptar el tamaño del tablero a las dimensiones reales. Se aprendió a calcular el espacio mínimo necesario en bytes, a reorganizar la memoria al insertar o eliminar filas/columnas (respetando la regla del 65%) y a evitar el uso de la STL, cumpliendo con las restricciones del desafío.
3. **Arquitectura Modular y Programación Estructurada:** La división del proyecto en múltiples archivos (.h y .cpp) facilitó la organización, el mantenimiento y la depuración del código. Esto evidenció la importancia de separar la lógica de bajo nivel (bits, memoria) de la lógica de juego (combinaciones, cascadas) y de la interfaz de usuario.
4. **Resolución de Problemas Complejos:** Se enfrentaron y superaron retos como las fichas repartidas entre dos bytes, las combinaciones simultáneas (horizontales y verticales), las cascadas automáticas y la reorganización estructural del tablero tras agregar o eliminar filas/columnas intermedias.
5. **Validación y Pruebas:** Como paso final, se identificó la necesidad de compilar y ejecutar el proyecto completo en Qt, verificando la correcta integración de todas las funciones (incluyendo reiniciarMarcador, mostrarTableroFichas y mostrarTableroBinario) y documentando con capturas las pruebas realizadas para garantizar el funcionamiento del sistema.

Finalmente, el uso de un repositorio Git con commits regulares permitió documentar fielmente la evolución de la solución. Este desafío no solo evaluó la capacidad de codificar, sino la de analizar, diseñar y optimizar soluciones informáticas bajo parámetros profesionales, dejando como aprendizaje principal la importancia de elegir la estructura de datos adecuada para resolver un problema de la manera más eficiente posible.